

CNN (Convolutional Neural Network)

What is CNN (Convolutional Neural Network)?

CNN ek **Deep Learning algorithm** hai jo specially **images, videos aur visual data** ko analyze karne ke liye use hota hai.

Ye image ke andar se **important features (edges, shapes, patterns)** automatically learn karta hai aur prediction karta hai.

Example:

- Input: Dog image
- Output: "Dog (99%)

Advantages of CNN

1. Automatic Feature Extraction

CNN khud features (edges, shapes, textures) learn karta hai — manual feature engineering nahi karni padti.

2. Fewer Parameters (compared to ANN)

Weight sharing aur local connectivity ki wajah se parameters kam hote hain → efficient model.

3. High Accuracy in Image Tasks

Image classification, detection me CNN bahut high accuracy deta hai.

4. Spatial Information Preserve karta hai

Image ka structure (2D relation) maintain rehta hai.

5. Better Performance on Large Data

Large image datasets par CNN best perform karta hai.

Disadvantages of CNN

1. High Computation Cost

Training ke liye GPU ki zarurat hoti hai.

2. Slow Training

Deep CNN models ko train karne me time lagta hai.

3. Large Dataset Requirement

Achha performance ke liye huge labeled dataset chahiye.

4. Complex Architecture

CNN ko design aur tune karna difficult hota hai (filters, layers, etc).

Limitations of CNN

1. Not good for non-image data

Tabular data (Excel type) ke liye CNN useful nahi hai.

2. Needs huge computing power

GPU/TPU ke bina training slow ho jati hai.

3. Overfitting risk

Small dataset par CNN easily overfit ho jata hai.

4. Black box model

Explain karna difficult hota hai ki model ne decision kaise liya.

Applications of CNN

1. Image Classification

- Cat vs Dog detection
- Object recognition

2. Medical Field

- Brain tumor detection
- X-ray analysis
- Cancer detection

3. Self Driving Cars

- Road detection
- Object detection (cars, people, signals)

4. Face Recognition

- Face unlock (mobile phones)
- Attendance systems

5. OCR (Optical Character Recognition)

- Handwritten digit recognition
- Document scanning apps

6. Agriculture

- Plant disease detection
- Crop quality analysis

7. Satellite Image Analysis

- Weather prediction
- Land use classification

8. Emotion Detection

- Happy, sad, angry face detection

Interview One-Liner

CNN is a deep learning model used for image processing that automatically extracts features using convolution operations and gives high accuracy in visual tasks.

CNN Architecture (Full Flow)

CNN ka architecture ek **step-by-step pipeline** hota hai jisme image ko gradually simple se complex features me convert kiya jata hai.

Overall CNN Architecture

```
Input Image
  ↓
Convolution Layer
  ↓
Activation (ReLU)
  ↓
Pooling Layer
  ↓
Convolution Layer (repeat)
  ↓
Activation (ReLU)
  ↓
Pooling Layer
  ↓
Flatten Layer
  ↓
Fully Connected (Dense Layer)
  ↓
Output Layer (Softmax / Sigmoid)
```

Step-by-Step Explanation

1. Input Layer

Raw image CNN ko input diya jata hai.

Example:

```
28 × 28 × 1 (grayscale)
224 × 224 × 3 (RGB)
```

Image pixel values me convert hoti hai.

2. Convolution Layer (Core of CNN)

Yahi CNN ka brain hota hai.

Kaam:

- Image se **features extract** karna
- Jaise edges, corners, shapes

How?

Filter / Kernel use hota hai:

Image + Filter $\rightarrow$ Feature Map

Example:

- Vertical edge detector
 - Horizontal edge detector
-

3. Activation Function (ReLU)

Non-linearity add karta hai

ReLU formula:

$$f(x) = \max(0, x)$$

Why?

- Negative values remove
 - Model ko complex patterns seekhne deta hai
-

4. Pooling Layer

Image size reduce karta hai (downsampling)

Types:

- Max Pooling (most used)
- Average Pooling

Example:

2x2 block → max value pick

Benefits:

- Computation reduce
 - Overfitting control
 - Important features retain
-

5. Repeated Conv + Pooling

CNN me ek hi layer nahi hoti.

Structure:

Conv → ReLU → Pool
Conv → ReLU → Pool

Early layers:

- edges detect

Deep layers:

- face, object detect
-

6. Flatten Layer

2D feature map ko 1D vector me convert karta hai

Example:

Feature Map:
 $7 \times 7 \times 64$

↓

Flatten:

3136×1 vector

7. Fully Connected Layer (Dense Layer)

Ye actual decision maker hota hai

- All neurons connected
- Final reasoning karta hai

Example:

Features → Dense Layer → Class prediction

8. Output Layer

Final prediction deta hai

Types:

Softmax (Multi-class)

Example:

- Cat
- Dog
- Horse

Cat: 70%

Dog: 20%

Horse: 10%

Sigmoid (Binary)

Example:

- Cancer / No Cancer

0 → No

1 → Yes

Import libraries

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
```

```
# CNN Model Design
```

```
model = Sequential()
```

```
# Step 1: Convolutional Layer
```

```
model.add(Conv2D(  
    filters=32,      # number of filters  
    kernel_size=(3,3),  # filter size  
    activation='relu',  # activation function  
    input_shape=(28,28,1) # input shape (MNIST grayscale images)  
))
```

```
# Step 2: Pooling Layer
```

```
model.add(MaxPooling2D(pool_size=(2,2)))
```

```
# Step 3: Flatten Layer
```

```
model.add(Flatten())
```

```
# Step 4: Dense Layers
```

```
model.add(Dense(128, activation='relu')) # hidden dense layer  
model.add(Dense(64, activation='relu')) # another hidden dense layer  
model.add(Dense(10, activation='softmax')) # output layer (10 classes)
```

```
# Step 5: Compile Model
```

```
model.compile(  
    optimizer='adam',  
    loss='categorical_crossentropy',  
    metrics=['accuracy']
```


)

Summary of the model

model.summary()

Life cycle CNN

1. Data Collection

|



2. Data Loading

|



3. Data Cleaning

├— Check Folder Structure

├— Count Images

├— Visualize Images

├— Remove Corrupted Images

├— Remove Duplicate Images

├— Fix Wrong Labels

├— Remove Poor Quality Images

└— Check Class Balance

|



4. Data Preprocessing

├— Resize Images

├— Normalize (0–255 → 0–1)

├— Data Augmentation (Optional)

- └─ Cache (Optional)
- └─ Prefetch
- └─ Train / Validation Split

|



5. Build CNN Model

- └─ Input Layer
- └─ Conv2D
- └─ ReLU
- └─ MaxPooling2D
- └─ Conv2D
- └─ ReLU
- └─ MaxPooling2D
- └─ Flatten
- └─ Dense
- └─ Output Layer

|



6. Compile Model

- └─ Optimizer
- └─ Loss Function
- └─ Metrics

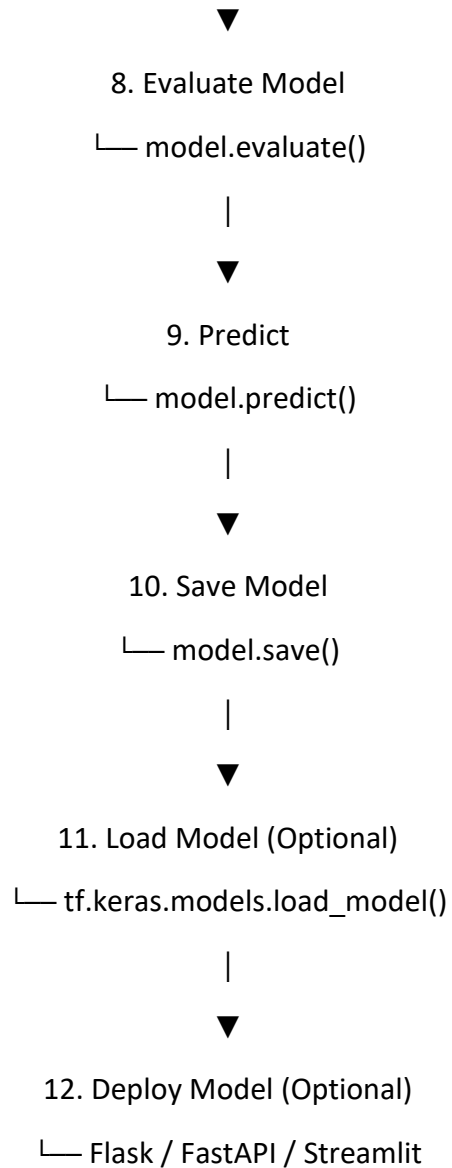
|



7. Train Model

- └─ model.fit()

|



Step 1 : Data Collection

Definition

Data Collection ka matlab model ke liye images collect karna.

Images aa sakti hain:

- Kaggle
- Roboflow
- Google Open Images
- Apna Camera
- Mobile
- Company Database

Step 2 : Data Loading

Definition

Data Loading ka matlab **dataset ko Python/TensorFlow me load karna**, taaki model us images ko read kar sake.

```
import tensorflow as tf          # TensorFlow import

# Training Dataset Load

train_dataset = tf.keras.preprocessing.image_dataset_from_directory(
    "../Dataset/carsdataset",    # Dataset Folder Path
    validation_split=0.2,        # 20% Validation Data
    subset="training",           # Training Dataset
    seed=42,                     # Same Split Every Time

    image_size=(224,224),       # Resize While Loading
    batch_size=32                # 32 Images per Batch
)

# Validation Dataset Load

validation_dataset = tf.keras.preprocessing.image_dataset_from_directory(
```

)

- image size=(224,224) → Loading ke time resize.

System Library

```
dataset_path = "../Dataset/carsdataset"           # Dataset Path
print(os.listdir(dataset_path))                   # Class Folder
Names
```

Output

```
['Bus', 'Car']
```

3.2 Count Images

```
import os
for folder in os.listdir(dataset_path):
    folder_path = os.path.join(dataset_path, folder)
    if os.path.isdir(folder_path):
        print(folder, ":", len(os.listdir(folder_path)))
```

Output

```
Bus : 2362
Car : 2362
```

3.3 Visualize Images

```
import matplotlib.pyplot as plt
from PIL import Image
import os

folder = "../Dataset/carsdataset/Bus"           # Class Folder
image = os.listdir(folder)[0]                   # First Image
img = Image.open(os.path.join(folder, image))
plt.imshow(img)
plt.axis("off")
plt.show()
```

Check:

- Blur Image
 - Wrong Image
 - Black Image
 - Cropped Image
-

3.4 Check Corrupted Images

```
from PIL import Image
import os

for folder in os.listdir(dataset_path):

    folder_path = os.path.join(dataset_path, folder)

    for image in os.listdir(folder_path):

        image_path = os.path.join(folder_path, image)

        try:
            img = Image.open(image_path)
            img.verify()                                # Verify Image

        except:
            print("Corrupted :", image_path)
```

Agar image corrupt hogi to uska path print ho jayega.

3.5 Check Duplicate Images

```
import os
import hashlib

hashes = {}

for folder in os.listdir(dataset_path):

    folder_path = os.path.join(dataset_path, folder)

    for image in os.listdir(folder_path):

        image_path = os.path.join(folder_path, image)
```

```
with open(image_path, "rb") as file:

    file_hash = hashlib.md5(file.read()).hexdigest()

if file_hash in hashes:

    print("Duplicate :", image_path)

else:

    hashes[file_hash] = image_path
```

Same image multiple times hogi to print ho jayegi.

3.6 Check Wrong Labels

Iska automatic code generally nahi hota.

Manual check karte hain.

Example

Bus Folder

↓

Car Image

Ya

Car Folder

↓

Bike Image

Aise images ko correct folder me move karte hain.

3.7 Check Poor Quality Images


```
from PIL import Image
import matplotlib.pyplot as plt

img = Image.open("../Dataset/carsdataset/Bus/bus1.jpg")

plt.imshow(img)

plt.axis("off")
```

Check:

- Blur
- Dark
- Over Bright
- Half Object
- Noise

Poor quality image ho to remove kar dete hain.

3.8 Check Class Balance

```
for folder in os.listdir(dataset_path):

    folder_path = os.path.join(dataset_path, folder)

    print(folder, len(os.listdir(folder_path)))
```

Output

Bus : 2362

Car : 2362

Agar bahut difference ho, jaise:

Bus : 5000

Car : 200

to dataset **imbalanced** hai.

Final Data Cleaning Flow

```
Check Folder Structure
    ↓
Count Images
    ↓
Visualize Images
    ↓
Check Corrupted Images
    ↓
Check Duplicate Images
    ↓
Check Wrong Labels
    ↓
Check Poor Quality Images
    ↓
Check Class Balance
    ↓
Clean Dataset
```

Placement Tip: Real projects me har baar duplicate ya corrupted image ka code nahi likha jata. Pehle **folder structure, image count aur visualization** ki jati hai. Agar dataset me issue dikhe, tab duplicate/corrupted image checks chalaye jate hain. Ye workflow industry me common hai.

Step 4 : Data Preprocessing

Definition

Data Preprocessing ka matlab clean images ko CNN ke input format me convert karna.

4.1 Resize Images

```
# Loading ke time hi Resize

train_dataset =
tf.keras.preprocessing.image_dataset_from_directory(

    "../Dataset/carsdataset",

    validation_split=0.2,

    subset="training",

    seed=42,
```

```
        image_size=(224,224),          # Resize All Images
        batch_size=32
    )
```

✓ Sabhi images

224 × 224

me convert ho jayengi.

4.2 Normalize Images

```
# Pixel Value

normalization_layer = tf.keras.layers.Rescaling(1./255)

train_dataset = train_dataset.map(
    lambda x, y: (normalization_layer(x), y)
)

validation_dataset = validation_dataset.map(
    lambda x, y: (normalization_layer(x), y)
)
```

Before

0 - 255

After

0 - 1

4.3 Data Augmentation (Optional)

```
data_augmentation = tf.keras.Sequential([
```

```
tf.keras.layers.RandomFlip("horizontal"),  
tf.keras.layers.RandomRotation(0.2),  
tf.keras.layers.RandomZoom(0.2)  
])
```

Model ko har epoch me images ke naye variations milte hain.

4.4 Cache (Optional)

```
train_dataset = train_dataset.cache()  
validation_dataset = validation_dataset.cache()
```

Dataset RAM me store hota hai.

Training fast hoti hai.

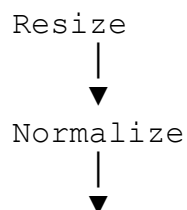
4.5 Prefetch

```
AUTOTUNE = tf.data.AUTOTUNE  
train_dataset = train_dataset.prefetch(AUTOTUNE)  
validation_dataset = validation_dataset.prefetch(AUTOTUNE)
```

GPU training aur data loading ek saath chalte hain.

Training aur fast hoti hai.

Final Data Preprocessing Flow



Data Augmentation (Optional)



Cache (Optional)



Prefetch



Ready For CNN

Final Code (Ek Sath)

```
# Normalize

normalization_layer = tf.keras.layers.Rescaling(1./255)

train_dataset = train_dataset.map(
    lambda x, y: (normalization_layer(x), y)
)

validation_dataset = validation_dataset.map(
    lambda x, y: (normalization_layer(x), y)
)

# Data Augmentation

data_augmentation = tf.keras.Sequential([

    tf.keras.layers.RandomFlip("horizontal"),

    tf.keras.layers.RandomRotation(0.2),

    tf.keras.layers.RandomZoom(0.2)

])

# Cache

train_dataset = train_dataset.cache()

validation_dataset = validation_dataset.cache()

# Prefetch

AUTOTUNE = tf.data.AUTOTUNE
```

```
train_dataset = train_dataset.prefetch(AUTOTUNE)

validation_dataset = validation_dataset.prefetch(AUTOTUNE)
```

Step 5 : Build CNN Model

Definition

Build Model ka matlab CNN ki architecture banana, jisme decide karte hain ki kitni layers hongi aur har layer ka kaam kya hoga.

Code

```
import tensorflow as tf

# CNN Model
model = tf.keras.Sequential([

    # Input Layer
    tf.keras.layers.Input(shape=(224,224,3)),          # 224x224 RGB Image

    # Convolution Block - 1
    tf.keras.layers.Conv2D(32, (3,3), activation="relu"), # 32 Filters
    tf.keras.layers.MaxPooling2D((2,2)),                # Down Sampling

    # Convolution Block - 2
    tf.keras.layers.Conv2D(64, (3,3), activation="relu"), # 64 Filters
    tf.keras.layers.MaxPooling2D((2,2)),

    # Convolution Block - 3
    tf.keras.layers.Conv2D(128, (3,3), activation="relu"), # 128 Filters
    tf.keras.layers.MaxPooling2D((2,2)),

    # Convert Feature Map → 1D
    tf.keras.layers.Flatten(),

    # Fully Connected Layer
    tf.keras.layers.Dense(128, activation="relu"),

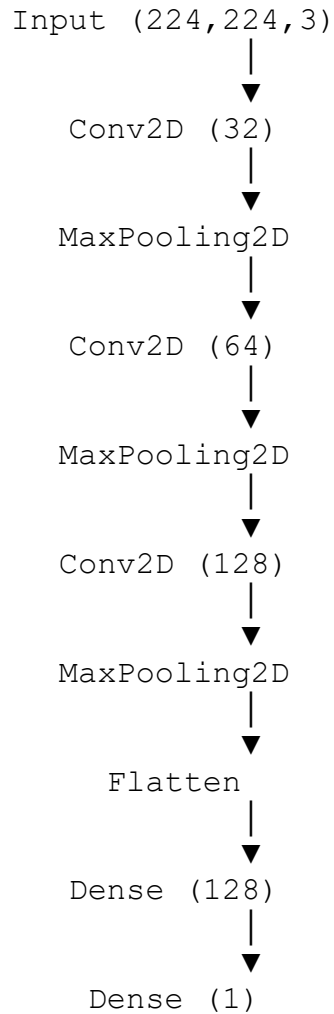
    # Output Layer (Binary Classification)
    tf.keras.layers.Dense(1, activation="sigmoid")

])
```

Model Summary

```
model.summary()
```

Output (Approx.)



Layer Purpose

Layer	Purpose
Input	Image Input
Conv2D	Features Extract (Edges, Shapes, Objects)
ReLU	Non-linearity
MaxPooling2D	Reduce Feature Map Size
Flatten	2D → 1D Convert
Dense	Learn Features
Output	Final Prediction

Binary vs Multi-Class

Binary Classification (2 Classes)

```
tf.keras.layers.Dense(1, activation="sigmoid")
```

Example:

- Cat vs Dog
- Car vs Bus
- Healthy vs Diseased

Multi-Class Classification (3+ Classes)

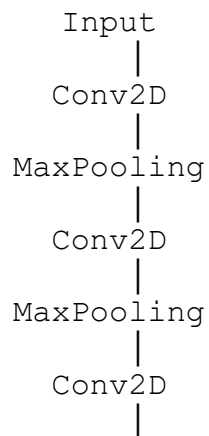
```
tf.keras.layers.Dense(5, activation="softmax")
```

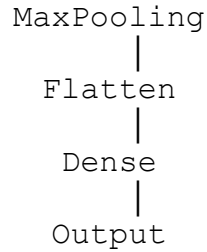
Example:

- Car
- Bus
- Bike
- Truck
- Van

Placement Tip

Ek basic CNN model ke liye ye architecture yaad rakho:





Step 6 : Compile Model

Definition

Compile Model ka matlab model ko batana ki **kaise seekhna hai, error kaise calculate karna hai, aur performance kaise measure karni hai.**

Code

```
model.compile(  
    optimizer="adam",                # Model ka learning  
    algorithm                         
    loss="binary_crossentropy",      # Binary Classification ke  
    liye Loss Function                
    metrics=["accuracy"]            # Performance Measure  
)
```

Compile Parameters

1. Optimizer

`optimizer="adam"`

Model ke weights update karta hai.

Most Common Optimizers:

Adam	Most Used
SGD	

RMSprop
Adagrad
AdamW

2. Loss Function

Binary Classification

```
loss="binary_crossentropy"
```

Example

Cat vs Dog

Car vs Bus

Healthy vs Diseased

Multi-Class Classification

```
loss="categorical_crossentropy"
```

OR

```
loss="sparse_categorical_crossentropy"
```

Example

Car

Bus

Bike

Truck

Van

3. Metrics

```
metrics=["accuracy"]
```

Accuracy show karega.

Aur bhi metrics use kar sakte ho.

```
metrics=[  
    "accuracy",  
    tf.keras.metrics.Precision(),  
    tf.keras.metrics.Recall()  
]
```

Binary Classification

```
model.compile(  
    optimizer="adam",  
    loss="binary_crossentropy",  
    metrics=["accuracy"]  
)
```

Multi-Class Classification

```
model.compile(  
    optimizer="adam",  
    loss="sparse_categorical_crossentropy",  
    metrics=["accuracy"]  
)
```

Regression

```
model.compile(  
    optimizer="adam",  
    loss="mse",
```

```
metrics=["mae"]  
)
```

Interview Tip: Agar interviewer puche "**Compile me kya hota hai?**", to ek line me bolo:

Compile step me hum model ko optimizer, loss function aur evaluation metrics batate hain, taaki training ke time model sahi tarike se learn kar sake.

Step 7 : Train Model

Definition

Train Model ka matlab model ko training data se **seekhana (learn karwana)**.

Code

```
history = model.fit(  
    train_dataset,                # Training Dataset  
    validation_data=validation_dataset, # Validation Dataset  
    epochs=10                     # Total Training Iterations  
)
```

history Object

```
history.history.keys()
```

Output

```
dict_keys([  
'loss',  
'accuracy',  
'val_loss',
```

```
'val_accuracy'  
])
```

Agar Graph banana ho

Accuracy Graph

```
import matplotlib.pyplot as plt  
  
plt.plot(history.history["accuracy"])  
  
plt.plot(history.history["val_accuracy"])  
  
plt.legend(["Train", "Validation"])  
plt.xlabel("Epoch")  
  
plt.ylabel("Accuracy")  
  
plt.show()
```

Loss Graph

```
plt.plot(history.history["loss"])  
  
plt.plot(history.history["val_loss"])  
  
plt.legend(["Train", "Validation"])  
  
plt.xlabel("Epoch")  
  
plt.ylabel("Loss")  
  
plt.show()
```

Training ke time 4 values sabse important hoti hain

Loss

Accuracy

Validation Loss

Validation Accuracy

Step 8 : Evaluate Model

Definition

Evaluate Model ka matlab trained model ki performance ko **Test Dataset ya Validation Dataset** par check karna.

Code

```
loss, accuracy = model.evaluate(
    validation_dataset          # Validation/Test Dataset
)
print("Loss :", loss)
print("Accuracy :", accuracy)
```

Step 9 : Predict

Definition

Predict ka matlab trained CNN model se **new image ka class predict** karwana.

Step 1 : New Image Load

```
from tensorflow.keras.utils import load_img, img_to_array

# New Image Load
img = load_img(
    "test.jpg",                                # New Image Path
    target_size=(224,224)                      # Same Size as Training Images
)
```

Step 2 : Image to Array

```
img = img_to_array(img) # Convert Image → NumPy Array
```

Step 3 : Normalize

```
img = img / 255.0 # Pixel Value 0-255 → 0-1
```

Step 4 : Add Batch Dimension

```
import numpy as np

img = np.expand_dims(img, axis=0) # (224,224,3) →
(1,224,224,3)
```

Step 5 : Prediction

```
prediction = model.predict(img)

print(prediction)
```

Binary Classification

```
if prediction[0][0] > 0.5:

    print("Car")

else:

    print("Bus")
```

Multi-Class Classification

```
prediction = model.predict(img)

class_index = np.argmax(prediction)

print(class_index)
```

Example

0 → Bus

1 → Car

2 → Bike

3 → Truck

Complete Prediction Code

```
from tensorflow.keras.utils import load_img, img_to_array
import numpy as np

# Load Image
img = load_img(
    "test.jpg",
    target_size=(224,224)
)

# Convert to Array
img = img_to_array(img)

# Normalize
img = img / 255.0

# Add Batch Dimension
img = np.expand_dims(img, axis=0)

# Prediction
prediction = model.predict(img)

# Binary Classification
if prediction[0][0] > 0.5:
    print("Car")
else:
    print("Bus")
```

Important

Prediction ke time bhi **wahi preprocessing** karni padti hai jo training ke time ki thi:

- Resize
- Normalize
- Batch Dimension

Agar training ke time preprocessing ki thi aur prediction ke time nahi ki, to model ki prediction galat ho sakti hai.

Step 10 : Save Model

Definition

Save Model ka matlab trained CNN model ko disk (laptop) me save karna, taaki baad me dobara train na karna pade.

Code

```
# Save Complete Model  
  
model.save("cnn_model.keras")
```

Custom Name

Tum apne hisaab se naam de sakte ho.

```
model.save("car_bus_classifier.keras")  
model.save("vehicle_detection.keras")  
model.save("my_cnn_model.keras")
```

Save Specific Folder

```
model.save("Models/cnn_model.keras")
```

Step 11 : Load Model

Definition

Load Model ka matlab pehle se save kiye hue model ko dobara memory me load karna, taaki bina training ke direct prediction kar sake.

Code

```
from tensorflow.keras.models import load_model

# Load Saved Model
model = load_model("cnn_model.keras")
```